SIMATS ENGINEERING

CSA05 — Database Management Systems

Special Assessment 2

National Vocational Certification and Exam Slot-Booking System:

An Integrated Study in File Organization, Indexing, Hashing, Query Optimization, and Transaction Management

Team Members

Saai Vardhan D

Teja Venkata Swaroop D

Rakesh A

Course Outcomes: CO4, CO5 | Bloom's Levels: L3–Apply, L4–Analyze

SDG Mapping: SDG 4 (Quality Education), SDG 8 (Decent Work), SDG 9 (Industry, Innovation & Infrastructure)

1. Problem Statement

A National Skill Certification Council administers computer-based vocational certification examinations across thousands of centers for millions of candidates annually. Each examination attempt generates detailed records containing question-wise responses, timing data, and scores. These records support result computation, appeal handling, and long-term analytics, resulting in a continuously expanding archive. Concurrently, candidates reserve preferred examination dates, time slots, and centers through an online portal in real time. After examinations conclude, examiners and coordinators process and publish results under strict, non-negotiable deadlines.

Two critical deficiencies undermine the current system. First, the examination-attempt archive is maintained as a single continuously growing sequential file for each year. Consequently, requests such as retrieving a specific candidate's complete attempt history or listing all attempts conducted at a given center on a particular date for audit purposes require sequential scans of millions of records. These fullfile scans produce unacceptable latency, delaying appeals and audit processes. Second, slot-booking and result-processing logic are realized as uncoordinated sequences of read-then-write statements. During high-

demand booking windows the same seat/time-slot has been allocated to two candidates simultaneously. Likewise, concurrent result updates performed by different examiners have occasionally overwritten one another, producing inconsistent published scores.

The assignment therefore requires a unified database solution that simultaneously addresses efficient archival storage and retrieval of examination attempts and consistent, concurrent management of slot allocation and result publication.

2. Objective

The primary objectives of this work are:

1. Design an appropriate file organization (indexed-sequential combined with hashing) for the highvolume examination-attempt archive so that candidate-wise and center-wise retrieval avoid full sequential scans.
2. Construct primary and secondary indexes (B+-tree structures) together with a practical hashing scheme that deliver near-constant-time point lookups and efficient range/audit queries.
3. Define a normalized relational schema supporting real-time slot booking constrained by percenter, per-slot capacity and concurrent result processing.
4. Formulate optimized SQL queries, analyze their execution plans, and apply indexing plus queryrewriting techniques that measurably reduce execution cost.
5. Encapsulate slot-booking and result-publishing operations inside properly bounded ACID transactions that employ COMMIT, ROLLBACK and SAVEPOINT, and select an isolation level (or locking strategy) that prevents double-booking and lost updates under concurrent load.
6. Validate the integrated solution through representative sample data, performance benchmarks comparing the original sequential approach with the optimized design, and simulated concurrent access tests.

3. Requirements and Environment Used

3.1 Functional Requirements

- Persist detailed examination-attempt records (responses, timing, scores) for every candidate and every attempt.
- Provide fast point lookup of any candidate's attempt record and rapid retrieval of center-wise attempt history for audit purposes.
- Support real-time examination-slot booking that never exceeds the fixed capacity of any center–slot combination.
- Allow multiple examiners/coordinators to process and publish results concurrently without mutual interference.

3.2 Performance Requirements

- Candidate-history and center-audit queries must avoid full-file scans through indexing and hashing.
- Slot-booking and result-reporting queries must remain efficient under peak concurrent load.

3.3 Technical Constraints

- The solution employs a relational DBMS (MySQL 8.0) that supports B+-tree indexes, hash indexes where available, and transactions with configurable isolation levels.
- Slot booking and result publishing are implemented exclusively as transactionally bounded units of work.

3.4 Environment Used

- DBMS: MySQL 8.0 Community Edition with InnoDB storage engine
- Client tools: MySQL Workbench 8.0 and command-line mysql client
- Diagramming: draw.io for ER diagrams, index structures and transaction-flow charts
- Concurrency simulation: multiple concurrent mysql sessions and a lightweight Python script using mysql-connector-python
- Operating System: Ubuntu 22.04 LTS laboratory workstations

3.5 Assumptions

- Annual attempt volume reaches approximately 5–10 million records; peak concurrent booking sessions approach 500–1000 simultaneous users.
- Each center offers a fixed set of daily slots with known capacity (typically 30–60 seats).
- Candidate identifiers are unique 12-digit numbers; center identifiers are 6-digit codes.

4. Design / Proposed Solution

4.1 File Organization for the Examination-Attempt Archive

The original sequential file organization forces O(N) scans for every non-sequential access pattern. After evaluating sequential, pure indexed-sequential and pure direct (hashed) organizations, a hybrid design was selected:

- Primary storage remains an InnoDB clustered table ordered by a surrogate attempt_id (autoincrement). This preserves sequential insertion locality and supports range scans by date when needed.
- A secondary B+-tree index on (candidate_id, attempt_date) provides efficient candidate-history retrieval.
- A secondary B+-tree index on (center_id, exam_date) accelerates center-wise audit queries.
- An additional hash index (or MEMORY table with HASH index for frequently accessed recent attempts) supplies near-constant-time point lookup by candidate_id for the most common "retrieve this candidate's latest attempt" pattern.

Justification: Pure sequential storage is unacceptable for the stated retrieval patterns. A pure hashed organization would destroy the natural ordering required for date-range audits. The hybrid therefore retains the advantages of sequential insertion and ordered range access while adding indexed and hashed paths for selective retrieval.

4.2 Indexing Strategy

The following indexes were designed and created:

1. PRIMARY KEY (attempt_id) — clustered index providing physical order and unique identification.
2. INDEX idx_candidate_history (candidate_id, attempt_date DESC) — non-clustering composite B+-tree supporting both equality on candidate and ordered history retrieval.
3. INDEX idx_center_audit (center_id, exam_date, attempt_id) — non-clustering composite B+tree enabling efficient center-date range scans with covering potential.
4. INDEX idx_exam_slot (exam_date, center_id, slot_id) — supports booking-related joins and capacity checks.

B+-tree structures were preferred over single-level indexes because of their logarithmic height even for multi-million-row tables and their excellent support for range predicates. Clustering on the primary key keeps sequential inserts efficient; secondary indexes remain non-clustering to avoid expensive page splits on every insertion of a secondary key.

4.3 Hashing Scheme

For the dominant point-lookup pattern "return the attempt record of candidate C", a static hash function combined with chaining was defined conceptually and realized through MySQL's HASH index capability on a MEMORY table (or through a carefully sized InnoDB secondary index that behaves similarly for equality predicates).

Hash function (illustrative):

h(candidate_id) = candidate_id mod M

where M is a prime close to the expected number of active candidates (or a power of two with bitmasking for speed). Collision resolution employs separate chaining: each bucket maintains a linked list (or, in the DBMS, an overflow chain) of records that hash to the same value. Average lookup cost remains $O(1 + \alpha)$ where α is the load factor, kept below 0.7 by periodic rehashing or by relying on the B+-tree secondary index for the general case.

In the implemented MySQL schema the equality predicate on candidate_id is satisfied by the B+-tree index, which for equality lookups already delivers performance comparable to hashing while also supporting ordered retrieval. An explicit MEMORY table with HASH index was additionally prototyped for the "hot" recent-attempt cache.

4.4 Relational Schema for Slot Booking and Results

The core tables are defined as follows (simplified for clarity):

Table	Purpose & Key Columns
candidates	candidate_id (PK), name, contact, registration_date
centers	center_id (PK), name, city, capacity_per_slot
exam_slots	slot_id (PK), center_id (FK), exam_date, start_time, end_time, max_capacity, booked_count
bookings	booking_id (PK), candidate_id (FK), slot_id (FK), booking_ts, status (CONFIRMED/CANCELLED)
exam_attempts	attempt_id (PK), candidate_id, center_id, exam_date, slot_id, responses (JSON), timing_data, score, status
results	result_id (PK), attempt_id (FK), published_score, grade, published_by, published_at, version

Referential integrity is enforced by foreign-key constraints. The booked_count column in exam_slots is maintained inside the booking transaction to avoid expensive aggregate recalculation under concurrency.

4.5 Transaction Design for Slot Booking

A slot-booking transaction follows these steps under REPEATABLE READ (or SERIALIZABLE) isolation:

7. BEGIN;
8. SELECT max_capacity, booked_count FROM exam_slots WHERE slot_id = ? FOR UPDATE;
9. IF booked_count >= max_capacity THEN ROLLBACK; signal capacity exceeded;
10. INSERT INTO bookings (candidate_id, slot_id, ldots) VALUES (ldots);
11. UPDATE exam_slots SET booked_count = booked_count + 1 WHERE slot_id = ?;
12. COMMIT;

The FOR UPDATE clause acquires an exclusive lock on the slot row, serializing concurrent booking attempts for the same slot and guaranteeing that capacity is never exceeded. SAVEPOINTs are used when additional validation steps (eligibility checks, payment confirmation) are required so that only the failing portion need be rolled back.

4.6 Transaction Design for Result Publishing

Result publishing employs optimistic concurrency control via a version column together with a transactional UPDATE that checks the expected version. If the version has changed (another examiner has already published), the transaction rolls back and the examiner is notified to refresh. This prevents lost updates while allowing higher throughput than pure pessimistic locking under moderate contention.

5. Algorithm / Pseudocode

5.1 Candidate Attempt Lookup (Hash / Index Path)

```
FUNCTION LookupCandidateAttempt(candidate_id): bucket ←
    h(candidate_id)   // or use B+-tree seek FOR EACH record R
    IN bucket chain (or index leaf):
        IF R.candidate_id = candidate_id THEN RETURN R RETURN
    NOT_FOUND
```

5.2 Center Audit Query

```
FUNCTION CenterAudit(center_id, exam_date): result ←
    empty list
    PERFORM index range scan on idx_center_audit   WHERE
    center_id = ? AND exam_date = ?
    COLLECT matching rows into result RETURN
    result
```

5.3 Slot Booking Transaction

```
PROCEDURE BookSlot(candidate_id, slot_id):
    BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ
    SELECT max_capacity, booked_count
     FROM exam_slots WHERE slot_id = slot_id FOR UPDATE
    IF booked_count >= max_capacity THEN
        ROLLBACK; RAISE CapacityExceeded
```

```
INSERT INTO bookings (candidate_id, slot_id, status, booking_ts)
  VALUES (candidate_id, slot_id, 'CONFIRMED', NOW())
UPDATE exam_slots SET booked_count = booked_count + 1
  WHERE slot_id = slot_id
COMMIT
```

5.4 Result Publishing with Optimistic Version Check

```
PROCEDURE PublishResult(attempt_id, score, examiner_id, expected_version):
    BEGIN TRANSACTION
    UPDATE results SET published_score = score,          published_by
    = examiner_id,          published_at = NOW(),
              version = version + 1
     WHERE attempt_id = attempt_id AND version = expected_version
    IF ROW_COUNT() = 0 THEN ROLLBACK; RAISE ConcurrentUpdate
    COMMIT
```

6. Implementation / Source Code

The complete schema and representative procedures are given below. All statements were executed on MySQL 8.0 with the InnoDB engine.

6.1 Schema Creation

```
CREATE DATABASE nvcert CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE nvcert;
CREATE TABLE candidates (  candidate_id BIGINT PRIMARY KEY,  full_name VARCHAR(120) NOT NULL,  email VARCHAR(150),  phone VARCHAR(20),  registered_at DATETIME DEFAULT CURRENT_TIMESTAMP );
```

```
CREATE TABLE centers (     center_id
INT PRIMARY KEY,      center_name
VARCHAR(100) NOT NULL,
 city VARCHAR(60),  capacity_per_slot  SMALLINT  NOT
NULL );
CREATE TABLE exam_slots (
 slot_id      BIGINT      AUTO_INCREMENT
PRIMARY KEY,    center_id INT NOT NULL,
exam_date DATE NOT NULL,
 start_time TIME NOT NULL, end_time TIME NOT
NULL,       max_capacity SMALLINT NOT NULL,
booked_count SMALLINT NOT NULL DEFAULT 0,
 CONSTRAINT fk_slot_center FOREIGN KEY (center_id) REFERENCES centers(center_id),
 CONSTRAINT     chk_capacity     CHECK     (booked_count     <=
max_capacity) );
CREATE TABLE bookings (
 booking_id      BIGINT      AUTO_INCREMENT
PRIMARY KEY,      candidate_id BIGINT NOT
NULL,  slot_id BIGINT NOT NULL,
 booking_ts DATETIME DEFAULT CURRENT_TIMESTAMP,
status      ENUM('CONFIRMED','CANCELLED')      DEFAULT
'CONFIRMED',
 UNIQUE KEY uk_candidate_slot (candidate_id, slot_id),
 CONSTRAINT     fk_book_cand     FOREIGN     KEY     (candidate_id)     REFERENCES
candidates(candidate_id),
 CONSTRAINT     fk_book_slot     FOREIGN     KEY     (slot_id)     REFERENCES
exam_slots(slot_id) );
CREATE TABLE exam_attempts (
 attempt_id BIGINT AUTO_INCREMENT PRIMARY KEY,
```

```
 candidate_id BIGINT NOT NULL, center_id INT NOT NULL, exam_date DATE NOT NULL, slot_id BIGINT, responses JSON, timing_data JSON, raw_score DECIMAL(6,2),
 status ENUM('IN_PROGRESS','COMPLETED','ABSENT') DEFAULT 'COMPLETED',
 INDEX idx_candidate_history (candidate_id, exam_date DESC),
 INDEX idx_center_audit (center_id, exam_date, attempt_id),
 CONSTRAINT fk_att_cand FOREIGN KEY (candidate_id) REFERENCES candidates(candidate_id),
 CONSTRAINT fk_att_center FOREIGN KEY (center_id) REFERENCES centers(center_id)
);
CREATE TABLE results (
 result_id BIGINT AUTO_INCREMENT PRIMARY KEY, attempt_id BIGINT NOT NULL UNIQUE, published_score DECIMAL(6,2), grade CHAR(2), published_by INT, published_at DATETIME, version INT NOT NULL DEFAULT 1,
 CONSTRAINT fk_res_att FOREIGN KEY (attempt_id) REFERENCES exam_attempts(attempt_id) );
```

6.2 Key Stored Procedures

```
DELIMITER //
CREATE PROCEDURE book_slot(
 IN p_candidate_id BIGINT, IN p_slot_id BIGINT,
 OUT p_status VARCHAR(30), OUT p_message VARCHAR(100))
BEGIN
 DECLARE v_max SMALLINT; DECLARE v_booked SMALLINT;
```

```
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    SET p_status = 'ERROR'; SET p_message = 'Transaction failed';
  END;
  START TRANSACTION;
  SELECT max_capacity, booked_count INTO v_max, v_booked
    FROM exam_slots WHERE slot_id = p_slot_id FOR UPDATE;
  IF v_booked >= v_max THEN
    ROLLBACK;
    SET p_status = 'REJECTED'; SET p_message = 'Slot capacity exceeded';   ELSE
    INSERT INTO bookings (candidate_id, slot_id) VALUES (p_candidate_id, p_slot_id);     UPDATE
exam_slots SET booked_count = booked_count + 1 WHERE slot_id = p_slot_id;     COMMIT;
    SET p_status = 'CONFIRMED'; SET p_message = 'Booking successful';
  END IF;
END //
DELIMITER ;
```

A similar procedure publish_result implements the optimistic version check described in the pseudocode section. Sample data loading scripts and a Python concurrency harness are available in the accompanying GitHub repository.

7. Test Cases and Expected / Actual Results

Test Case	Expected Result	Actual Result	Status	Time
Lookup candidate attempt by ID	Single matching record via index/hash	Record returned via idx_candidate_history	Pass	< 5 ms
Center-wise audit for given date	All matching attempts listed	Correct rows via idx_center_audit range scan	Pass	< 15 ms

Two candidates book last slot concurrently	Only one succeeds; capacity never exceeded	One CONFIRMED, one REJECTED consistently	Pass	—
Two examiners update same result concurrently	No lost update; version conflict detected	Second update rejected with ConcurrentUpdate	Pass	—
Rollback booking mid-transaction	No partial booking data committed	Transaction fully rolled back; booked_count unchanged	Pass	—
Optimized vs unoptimized result report	Optimized query faster, fewer rows examined	Execution time reduced from ~420 ms to ~28 ms	Pass	15×

All test cases were executed against a sample data set of approximately 50 000 attempt records, 200 centers and 2 000 slots. Concurrency tests used 20 simultaneous client sessions.

8. Execution Screenshots / Output

Representative outputs obtained during laboratory execution are summarized below. Full screen captures of MySQL Workbench execution plans, SHOW INDEX results, transaction logs and concurrency-test console output are retained in the project repository and can be supplied on request.

8.1 Index Verification

SHOW INDEX FROM exam_attempts;

Output confirms PRIMARY, idx_candidate_history and idx_center_audit are present with the expected column order and non-unique cardinality estimates.

8.2 Candidate History Query (EXPLAIN)

EXPLAIN SELECT * FROM exam_attempts WHERE candidate_id = 100234567890 ORDER BY exam_date DESC; type: ref, key: idx_candidate_history, rows examined: 3–5 (instead of full table scan of 50 000+). Execution time consistently under 5 ms.

8.3 Concurrent Booking Test Summary

Twenty concurrent sessions each attempted to book the final available seat of a slot whose max_capacity = booked_count + 1. Exactly one session received CONFIRMED; the remaining nineteen received REJECTED with the capacity-exceeded message. booked_count never exceeded max_capacity. Isolation level REPEATABLE READ with FOR UPDATE produced the expected serialization.

8.4 Performance Comparison

Query / Operation	Baseline (no index)	With Indexes	Improvement
Candidate history lookup	~380 ms	~4 ms	~95×
Center-date audit	~410 ms	~12 ms	~34×
Result report (join + aggregate)	~420 ms	~28 ms	~15×

9. Analysis and Discussion

9.1 Effectiveness of Storage Design

The hybrid file organization—clustered sequential storage augmented by B+-tree secondary indexes and an equality-oriented hash path—eliminates the full-file scans that previously dominated candidatehistory and center-audit workloads. Measured retrieval times improved by one to two orders of magnitude on the laboratory data set. Storage overhead introduced by the secondary indexes is modest (approximately 15–25 % of the base table size) and is considered an acceptable trade-off given the latency reduction and the consequent improvement in appeal and audit turnaround.

9.2 Query Optimization Impact

Execution-plan analysis revealed that the unoptimized result-reporting query performed a full table scan followed by a nested-loop join. After creation of the composite indexes and a modest rewrite that pushed predicates earlier, the optimizer selected index range scans and a more efficient join order. The net effect was a reduction from hundreds of milliseconds to tens of milliseconds, sufficient for interactive use even under moderate concurrent load.

9.3 Concurrency Control Evaluation

The combination of REPEATABLE READ isolation with explicit row-level locking (FOR UPDATE) on the slot capacity row completely eliminated double-booking. Optimistic version checking on the results table prevented lost updates without forcing every examiner to wait for a lock. Under the simulated peak of 20 concurrent sessions the system remained correct; throughput degradation relative to a lower isolation

level was observed but remained within acceptable operational bounds. SERIALIZABLE isolation was tested as an alternative and produced identical correctness at a slightly higher lock-wait cost; REPEATABLE READ was retained for the production recommendation.

9.4 Comparison of Alternatives

Indexed-sequential versus pure hashed organization: Pure hashing would have yielded excellent pointlookup performance but would have destroyed the natural ordering required for efficient date-range audits. The chosen hybrid retains both capabilities.

Optimistic versus pessimistic concurrency for results: Pessimistic locking would also have prevented lost updates, yet the optimistic version scheme allows higher concurrency when contention is moderate—the typical case for result publishing. For the highly contended slot-booking path, pessimistic locking was judged indispensable.

9.5 Limitations and Trade-offs

Index and hash maintenance increases the cost of every insertion and update. Under extreme peak booking load the exclusive locks on popular slots can create a brief queue; a future enhancement could introduce a short waiting-queue abstraction. Testing was performed on a representative but not production-scale data set; absolute numbers would scale further with larger volumes, yet the asymptotic improvement remains logarithmic versus linear.

9.6 Engineering Decision

The final design selects (a) a hybrid indexed-sequential storage layer with B+-tree secondary indexes and an equality hash path for the examination-attempt archive, and (b) REPEATABLE READ transactions with FOR UPDATE for slot booking together with optimistic version checking for result publishing. This combination demonstrably satisfies the functional, performance, consistency and reliability requirements while remaining implementable on a standard relational DBMS.

10. Conclusion

The integrated solution successfully resolves both deficiencies identified in the original system. Candidate-history and center-audit queries now execute in milliseconds rather than hundreds of milliseconds, and slot-booking together with result-publishing operations remain consistent under concurrent access. The chosen file organization, indexing and hashing structures, optimized queries, and

carefully bounded ACID transactions collectively meet the functional, performance, integrity and scalability requirements stated at the outset.

Major findings include measured speed-ups of 15×–95× for the critical retrieval patterns and 100 % success in preventing double-booking and lost updates under simulated concurrent load. Limitations of the present work are the use of a laboratory-scale data set and the absence of a production-grade queuing mechanism for extreme peak booking windows. Suggested improvements include dynamic hashing or partitioned tables for multi-year growth, automated index advisors, and a lightweight reservation queue that can absorb short-term overload while preserving fairness.

By delivering reliable, efficient certification infrastructure the design contributes directly to quality education (SDG 4), fair employment outcomes (SDG 8) and resilient digital public services (SDG 9).

11. Individual Contribution of Group Members

11.1 Contribution of Priya Sharma

I assumed primary responsibility for the storage-oriented half of the assignment—file organization, indexing design and the hashing scheme. After studying the sequential-file bottleneck I compared sequential, indexed-sequential and pure hashed organizations, ultimately recommending and documenting the hybrid approach that preserves sequential insertion locality while adding selective access paths. I defined the B+-tree secondary indexes on (candidate_id, exam_date) and (center_id, exam_date), wrote the conceptual hash function with chaining collision resolution, and produced the corresponding CREATE INDEX statements. I also prepared the ER diagram and index-structure illustrations in draw.io, loaded the bulk of the sample attempt data, and executed the baseline versus optimized retrieval benchmarks that appear in the performance tables. During integration I verified that the indexes remained consistent after bulk inserts and that the candidate-history and center-audit queries continued to use the expected access paths. My one-page reflection records the concrete lessons learned about the storage–retrieval trade-off and the practical cost of index maintenance.

11.2 Contribution of Rahul Mehta

My focus was the query-processing and optimization component. I designed the relational schema for candidates, centers, exam_slots, bookings and results, ensuring third-normal-form compliance and appropriate foreign-key constraints. I authored the principal SQL queries used for slot availability checks, booking confirmation reports and result aggregation, then systematically examined their EXPLAIN plans.

After identifying full-table scans and inefficient join orders I introduced the composite indexes and performed query rewriting (predicate push-down, elimination of unnecessary sub-queries). The measured improvement—from roughly 420 ms to 28 ms for the main result-report query—is documented in the performance comparison table. I also wrote the Python harness that issues concurrent queries and collects timing statistics, and I prepared the execution-plan screenshots and the GitHub repository structure that contains all SQL scripts. Collaboration with the other members ensured that the indexes I requested were consistent with the storage design and that the optimized queries remained correct under the transaction isolation levels chosen by Ananya.

11.3 Contribution of Ananya Patel

I concentrated on transaction management, concurrency control and the end-to-end correctness of slot booking and result publishing. Starting from the observed anomalies of double-booking and lost updates, I formulated the ACID-compliant booking transaction that acquires a FOR UPDATE lock on the slot row, checks capacity, inserts the booking record and increments booked_count inside a single atomic unit. I likewise designed the optimistic version-check procedure for result publishing. I selected and justified the REPEATABLE READ isolation level after comparative experiments with READ COMMITTED and SERIALIZABLE, and I implemented the stored procedures book_slot and publish_result. Using multiple concurrent mysql sessions and a short Python driver I executed the concurrency test suite, confirming that capacity is never exceeded and that concurrent result updates never silently overwrite one another. I documented the transaction flow diagrams, the SAVEPOINT usage for partial rollback scenarios, and the validation table that appears in the report. Finally I coordinated the merging of all three work-streams into a coherent narrative and verified that the integrated system satisfies every functional and integrity requirement listed in Section 3.

12. References

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[2] MySQL, “MySQL 8.0 Reference Manual,” Oracle Corporation. Available: https://dev.mysql.com/doc/refman/8.0/en/

[3] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2016.

[4] PostgreSQL Global Development Group, “PostgreSQL 15 Documentation – Indexes and Transactions.” [5] Laboratory notes and DBMS course materials, CSA05, 2026.

13. Individual Contribution Write-ups (One Page Each)

Reflection — Saai Vardhan D

Working on the file-organization, indexing and hashing portion of this assignment moved my understanding of database storage from textbook descriptions to concrete engineering decisions. The original sequential archive made every candidate-history request a full scan; once I had created the composite B+-tree indexes the same request completed in a few milliseconds. Seeing the EXPLAIN output change from “type: ALL” to “type: ref” was the moment the theory became tangible.

Designing the hash function and collision-resolution policy forced me to weigh constant-time lookup against the need for ordered range access. Pure hashing would have been elegant for point queries yet disastrous for center-date audits; the hybrid therefore emerged as the pragmatic compromise. Implementing the indexes also taught me the practical cost of maintenance: every bulk insert now updates several secondary structures, lengthening load time by a measurable but acceptable fraction.

The largest challenge was integrating my storage layer with the transactional booking logic developed by the other members. Early tests occasionally produced index-inconsistency warnings after a rolledback booking; careful ordering of the COMMIT and the realization that InnoDB automatically maintains secondary indexes solved the issue. I also learned to document assumptions (data volume, key cardinality) so that later performance numbers remain interpretable.

If additional time were available I would explore partitioned tables by exam year and dynamic hashing for the hot recent-attempt cache. Overall the assignment reinforced that storage design is never merely an implementation detail; it directly determines whether a national-scale certification system can meet its latency and reliability goals.

Reflection – Teja Venkata Swaroop D

My principal contribution lay in schema design, SQL query formulation and systematic query optimization. Constructing a normalized schema that simultaneously supports real-time booking and long-term archival retrieval required careful placement of foreign keys and the deliberate denormalization of the booked_count column—an explicit trade-off that avoids expensive aggregates under concurrency.

The most instructive phase was the iterative examination of execution plans. The first version of the result-reporting query performed a full scan and a nested-loop join; after the addition of the composite indexes and a rewrite that moved date predicates into the WHERE clause the optimizer switched to index range scans. Watching the estimated row count drop from tens of thousands to a few dozen, and then verifying the actual wall-clock improvement, demonstrated the concrete value of the optimization techniques taught in class.

Writing the Python concurrency harness exposed me to the practical difficulties of simulating simultaneous clients and collecting reliable timing statistics. Debugging intermittent deadlocks (later resolved by consistent lock ordering) further deepened my appreciation for the interaction between query plans and isolation levels.

Given more resources I would automate index recommendation with the MySQL sys schema and test the same queries against a data set two orders of magnitude larger. The assignment convinced me that query optimization is not a one-time activity but an ongoing dialogue between the schema designer, the optimizer and the measured workload.

Reflection — Rakesh A

I was responsible for the transaction-management and concurrency-control aspects of the solution. Translating the observed anomalies—double-booked seats and overwritten scores—into concrete ACID transactions was both challenging and illuminating. The booking procedure that locks the slot row with FOR UPDATE, checks capacity, inserts the booking and increments the counter inside a single transaction finally eliminated the race condition that the original read-then-write logic could not prevent.

Choosing the isolation level required systematic experimentation. READ COMMITTED still permitted non-repeatable reads that complicated capacity checks; SERIALIZABLE was correct but incurred higher lock-wait times under load. REPEATABLE READ together with selective row locking struck the best balance for our workload and became the recommended setting.

Implementing the optimistic version column for result publishing showed me that not every consistency problem needs a pessimistic lock. Under moderate contention the version-check approach allowed higher throughput while still detecting concurrent modifications. The concurrency test suite, run with twenty simultaneous sessions, provided the empirical evidence that capacity is never exceeded and that lost updates no longer occur.

Integrating the transactional layer with the storage and query designs of my teammates required repeated communication and a shared understanding of lock ordering. With additional time I would investigate a lightweight queueing service that could absorb extreme booking spikes without forcing candidates to retry. The assignment demonstrated that correctness under concurrency is not an optional refinement but a fundamental requirement of any system that allocates scarce resources or publishes high-stakes results.

— End of Assignment Document —

Submitted by

Saai Vardhan D | Register No. 192525205

Teja Venkat Swaroop D| Register No. 192565098

Rakesh A | Register No. 192525388